

Cat vs Dog: The Permutation Way

Maria Camila Castaño Martínez

18/03/2026

For this project, I'll present an inefficient *esoteric algorithm* for solving a classical machine learning problem: image classification between cats and dogs. We know that modern approaches typically rely on Convolutional Neural Networks (CNNs).

1 Algorithm Name

Cat vs Dog: The Permutation Way

2 Type of Algorithm

Binary image classifier. The algorithm receives an image as input and outputs one of two labels:

- Cat
- Dog

The classification is performed using a *k-nearest neighbors (k-NN)* approach.

3 Dataset

The dataset used is the *Cats and Dogs Classification Dataset* available on Kaggle [1]. It contains:

- 24,998 images total (12,499 cat images and 12,499 dog images)

4 Hyper-parameters

The algorithm uses the following parameters:

- h, w : image height and width
- $p = 3hw$: number of pixel values (RGB image)
- n : number of training images
- k : number of nearest neighbors used for classification

All images are resized to:

$$64 \times 64 \times 3$$

which results in:

$$p = 12288$$

pixel values per image.

5 Algorithm Description

The algorithm consists of the following steps.

Step 0: Image Normalization

All images are resized to a fixed resolution of 64×64 pixels while preserving RGB channels.

Step 1: Image Flattening

The image is converted into a one-dimensional vector:

$$[x_1, x_2, x_3, \dots, x_p]$$

where each value corresponds to a pixel intensity in the range $[0, 255]$.

Step 2: Conversion to a Large Integer

The pixel vector is converted into a very large integer:

$$N = x_1 \cdot 256^0 + x_2 \cdot 256^1 + x_3 \cdot 256^2 + \dots + x_p \cdot 256^{p-1}$$

This step creates a numeric encoding of the entire image.

Step 3: Permutation Generation

The digits of N are used to generate a permutation of the pixel indices.

This permutation determines how the pixel positions will be rearranged.

Step 4: Pixel Rearrangement

Using the generated permutation, the image vector is reordered:

Original:

$$[x_1, x_2, x_3, \dots, x_p]$$

Permuted:

$$[x_{\pi(1)}, x_{\pi(2)}, x_{\pi(3)}, \dots, x_{\pi(p)}]$$

This step intentionally destroys the spatial structure of the image.

Step 5: Classification

The scrambled images are classified using a *k-nearest neighbors* classifier.
The classification is based on distance between the permuted pixel vectors.

6 Machine Requirements

The algorithm can run on a standard sequential machine. However, parallel computing could improve performance during the nearest neighbor comparisons, since distance computations between the test image and all training images can be performed independently. No special materials or hardware are required beyond any standard computational resources.

7 Time Complexity

The complexity can be analyzed by examining each step of the algorithm.

Pre-processing	Classification
$O(p)$	$O(np)$

Table 1: Time complexity of Pre-processing and Classification.

where:

- n is the number of training images
- p is the number of pixel values

8 Best and Worst Case Running Time

Worst case	Best case
$O(np)$	$O(np)$

Table 2: Time complexity of Pre-processing and Classification.

9 Why the Algorithm is Esoteric

This algorithm intentionally introduces unnecessary transformations that degrade classification performance. By mixing pixel positions before classification, the algorithm removes the spatial relationships that are essential for image recognition. This algorithm deliberately violates that principle, making it inefficient and ineffective by design.

10 References

- [1] Jikadara, B. (2024). Cats and Dogs Classification Dataset. Kaggle.com.
<https://www.kaggle.com/datasets/bhavikjikadara/dog-and-cat-classification-dataset/data>